

COMPLETE PROJECT GUIDE & TECHNICAL MANUAL

ARGUS

Closed-Loop, Evidence-Bound IoT Intrusion Detection
and Autonomous Response

Author: Raghav Anand

Program: MCA, CHRIST (Deemed to be University), Bangalore

Purpose: Viva · Project Presentation · Technical Interview · Resume Discussion

Repository branch: `claude/nifty-ramanujan-ruaew2`

Table of Contents

1. Project Overview

2. Problem Being Solved

3. Proposed Solution

4. Core Features

5. Complete System Architecture

6. End-to-End Workflow

7. Technology Stack

8. Frontend Deep Dive

9. Backend Deep Dive

10. Database

11. AI / ML Component

12. Security

13. Important Code Concepts

14. API / Data Flow

15. Deployment Architecture

16. Local Setup Guide

17. Production Deployment Guide

18. Testing

19. Troubleshooting

20. Limitations

21. Future Enhancements

22. Resume Explanation

23. Viva / Interview Preparation

1. Project Overview

ARGUS is a closed-loop, evidence-bound intrusion detection and autonomous response system built for small, unmanaged IoT networks (homes, clinics, small shops — 5 to 50 devices, no dedicated security analyst). Unlike a conventional IDS that stops at raising an alert, ARGUS runs the full loop: **detect** → **correlate** → **assess risk** → **explain** → **decide** → **enforce** → **verify**, and feeds the verification outcome back into its own risk and behaviour models.

The project has two parallel, interchangeable data sources feeding an identical downstream pipeline: a deterministic **synthetic simulation engine** for fast, repeatable demos and testing, and a **real Linux network-namespace testbed** that carries genuine packets captured with `scapy`. Both feed the same detection, evidence, response, and verification code — nothing downstream needs to know which source produced the data.

It ships as three runnable pieces: a Python/FastAPI backend, a React/TypeScript analyst console, and an optional real-Linux live-testbed extra. A production build is also deployed as a static console plus a lightweight Python serverless function on Vercel (Section 17).

2. Problem Being Solved

2.1 The operational gap

A compromised IoT device on a small, unmanaged network stays compromised indefinitely — it keeps scanning, beaconing to command-and-control infrastructure, or participating in a botnet — because nobody is watching, and even systems that do watch typically only raise an alert that nobody reads.

2.2 The evidence for this being a real, documented gap

A 2026 systematic review (Sallam, El Barachi & Li, published in *IoT*, DOI 10.3390/iot7010016) examined 32 recent IoT intrusion detection proposals and found that 29 of them were evaluated offline only, that none were validated under realistic multi-node conditions, and that mitigation (the ability to actually act on a detection) was "often absent" entirely. This project's central design decision — build the response and verification loop, not another classifier — responds directly to that documented gap.

2.3 The specific research question this project asks

On a network with no human operator, can an automated system detect a compromised device, decide whether to act, act on the real network, prove afterward why it acted, and not break the network's legitimate function — and how would anyone know, concretely, whether it succeeded?

3. Proposed Solution

3.1 Simple explanation

Think of ARGUS as a security guard for a small IoT network that doesn't just point at something suspicious and shout — it investigates, decides what to do about it (ignore it, watch it more closely, cut its network access, or ask a human), does it, checks afterward whether that actually fixed the problem, and keeps a tamper-evident notebook explaining every decision it made, so anyone can check its homework later.

3.2 Technical explanation

ARGUS combines three detection tracks (declared-policy rule checks, hand-authored signature rules, and a calibrated Isolation Forest anomaly detector gated by an inductive conformal-prediction wrapper) feeding a correlator that groups related detections into incidents. Each incident is scored by a risk engine, explained with TreeSHAP attribution, and bound into a hash-chained evidence bundle before a safety-guard component (nine independent veto checks, evaluated in order) decides whether and how to respond, using a six-tier response ladder from passive observation up to full isolation. Every applied action carries an auto-expiring rollback timer. Post-response verification checks the outcome and feeds it back into the risk and behaviour models, closing the loop.

Why this design, not a bigger classifier: the project's own measured result (Section 11.6) is that classification accuracy barely matters if the system attached to the classifier can't or won't act — so the engineering effort went into the response, evidence, and safety machinery around detection, not into chasing a marginally higher F1 score.

4. Core Features

Each feature below is described the same way: what it is, why it exists, how it works, its inputs/processing/outputs, which frontend/backend piece implements it, how it touches the database, and any security-relevant note.

4.1 Device Registry & Enrollment

What	A bounded, policy-guarded process that adds a device to ARGUS's monitored fleet and captures its behavioural baseline.
Why	A device compromised <i>before</i> its baseline is captured would otherwise have the attack behaviour learned in as "normal."
How	New devices start in an <code>ENROLLING</code> state; enrollment can be refused on a declared-policy violation rather than blindly accepted.
Input / Output	Device metadata (type, declared policy) in; a <code>DeviceRow</code> with state <code>MONITORED</code> (or a documented refusal) out.
Backend module	<code>argus/registry/</code>
Frontend	Fleet tab (<code>console/src/pages/Fleet.tsx</code>)
DB	<code>devices</code> table

4.2 Behaviour Engine & Drift Monitor

What	Tracks each device's behavioural baseline and detects when it drifts (changes shape without being compromised).
Why	Conflating benign drift (e.g. a firmware update) with compromise is the classic failure mode of baseline-deviation detectors.
How	An online ADWIN (Adaptive Windowing) drift detector runs independently of the behaviour engine; a device in a detected drift state has its maximum response tier <i>capped</i> at passive <code>alert</code> — drift <i>suppresses</i> escalation, it never triggers it.
Backend module	<code>argus/behavior/</code> (uses the <code>river</code> library's ADWIN)
Frontend	Fleet tab shows a "drifting" badge and a suppressed-escalation notice per device.
DB	<code>devices.is_drifting</code>

4.3 Metadata-Only Feature Extraction

What	Converts raw flow records into seven feature groups: volume/shape, timing, periodicity, destination behaviour, DNS, TLS/JA4 fingerprint, and policy violation.
Why	Payload inspection is a permanent non-goal (privacy, and it doesn't work on encrypted traffic anyway); every feature group is justified by a specific attack behaviour it exposes.
How	Pure statistical/metadata computation per flow-window — no packet contents are ever read.
Backend module	<code>argus/features/extract.py</code>

4.4 Rule / Policy Detection

What	Zero-model detections: a device contacting something outside its declared policy allowlist, or matching an authored signature (e.g. DNS query-entropy threshold for tunnelling, scan fan-out threshold for Mirai-style recruitment, TLS/JA4 fingerprint mismatch for identity spoofing).
Why	Highest precision, free human-readable explanation, no training data needed.
Backend module	<code>argus/detect/rules.py</code>

4.5 Calibrated ML Detection (Isolation Forest + Conformal Gate)

What	An unsupervised anomaly detector giving coverage of attacks no rule was authored for.
Why	Rules only catch known shapes; ML covers the unknown.
How	A per-device-type Isolation Forest, isotonic-calibrated on a held-out split, wrapped in an inductive conformal-prediction gate (implemented from scratch) that produces a prediction set rather than a bare label; escalation past a passive alert requires the set to be a singleton at $\alpha=0.05$.
Backend module	<code>argus/detect/ml.py</code>
Security note	This is exactly the mechanism behind contribution claim C3 — see the IEEE paper (Section VII) for the formal guarantee it provides.

4.6 SHAP Attribution

What	Real per-feature TreeSHAP contribution values for every ML detection.
Why	An explanation that can't be independently re-derived from the data that produced it isn't evidence — it's decoration.
How	A <code>RandomForestClassifier</code> trained on the same labelled calibration split the conformal detector uses; TreeSHAP values are computed and stored directly in the evidence bundle.
Frontend	Incidents drawer — a diverging bar chart of feature contributions (<code>console/src/components/RiskBadge.tsx</code> 's <code>AttributionChart</code>).

4.7 Correlation & Risk Scoring

What	Groups related detections (same device/time window/chain position) into one incident; scores each incident on a 0–1 risk scale.
How	Risk is a weighted sum of five interpretable terms — severity, device criticality, calibrated confidence (discounted for a non-singleton conformal set), behavioural deviation, and blast radius (from the observed communication graph, weighted by neighbouring-device criticality). Weights are authored, defended by a $\pm 50\%$ sensitivity analysis, not learned.
Backend module	<code>argus/correlate/</code> , <code>argus/risk/</code>
DB	<code>incidents</code> , <code>risk_assessments</code>

4.8 Evidence Bundle & Replay

What	A hash-chained record of every autonomous decision, plus a mechanism that re-runs the decision from the recorded values and checks it matches.
Why	"Why did it act" needs a checkable answer, not a plausible-sounding one.
How	Every bundle stores the feature vector, detection signals, conformal set, SHAP attribution, risk terms, the guard's full decision trace, the resulting action, and a SHA-256 hash chained to the previous bundle. <code>argus/evidence/replay.py</code> re-runs the guard/ladder logic from those stored values against a fresh guard state and compares the outcome.
Frontend	Incidents drawer's "Replay decision" button, calling <code>POST /evidence/{id}/replay</code> live.
DB	<code>evidence_bundles</code>
Security	Requires the admin bearer token (state-changing in the sense that it exercises live decision logic, even though it never touches the network).

4.9 Safety Guard

What	Nine independent veto checks, evaluated in a fixed order, that must all pass before any action is taken.
Why	Safety cannot be an emergent property of the decision engine's own logic — it is a separate component with veto power that no code path can bypass.
Checks	(1) kill switch — file, env var, and API, all three checked; (2) dry-run default; (3) protected-device list (never overridden by risk score); (4) conformal singleton gate; (5) drift gate; (6) per-tier risk threshold; (7) irreversibility → requires approval; (8) rate limit (per-device and network-wide, per hour); (9) rollback timer (every action carries a TTL and auto-reverts unless renewed).
Backend module	<code>argus/respond/guard.py</code>
Frontend	Control tab shows kill-switch state and lets an authenticated user engage/disengage it — never the enforcement flag itself, on purpose.

4.10 Response Ladder & Enforcement

What	Six response tiers (observe, alert, rate_limit, block_destination, isolate, require_approval), each behind a single adapter interface.
How	The default adapter everywhere is <code>DryRunAdapter</code> (logs intent, touches nothing). A real <code>nftables</code> -backed adapter exists and is verified against actual socket connections, but is never wired in as a default — opt-in only.
Backend module	<code>argus/respond/ladder.py</code> , <code>argus/respond/adapters/</code>
DB	<code>actions</code>

4.11 Post-Response Verification

What	Checks at T+30s / +2m / +5m whether the threat stopped, the device recovered, and nothing legitimate broke.
How	Outcomes feed back: <code>collateral_damage</code> raises the action threshold for that device class; <code>not_contained</code> escalates if gates still pass; <code>contained</code> closes the loop.
Backend module	<code>argus/verify/</code>
DB	<code>verifications</code>

4.12 Analyst Console

What	A React/TypeScript single-page app with three screens: Fleet, Incidents (with live evidence, SHAP chart, and replay), and Control.
Accessibility	Keyboard-operable incident rows, a real ARIA dialog with focus management and Escape-to-close, <code>aria-live</code> status regions — verified with browser automation, not just code review.
Frontend module	<code>console/src/</code>

5. Complete System Architecture

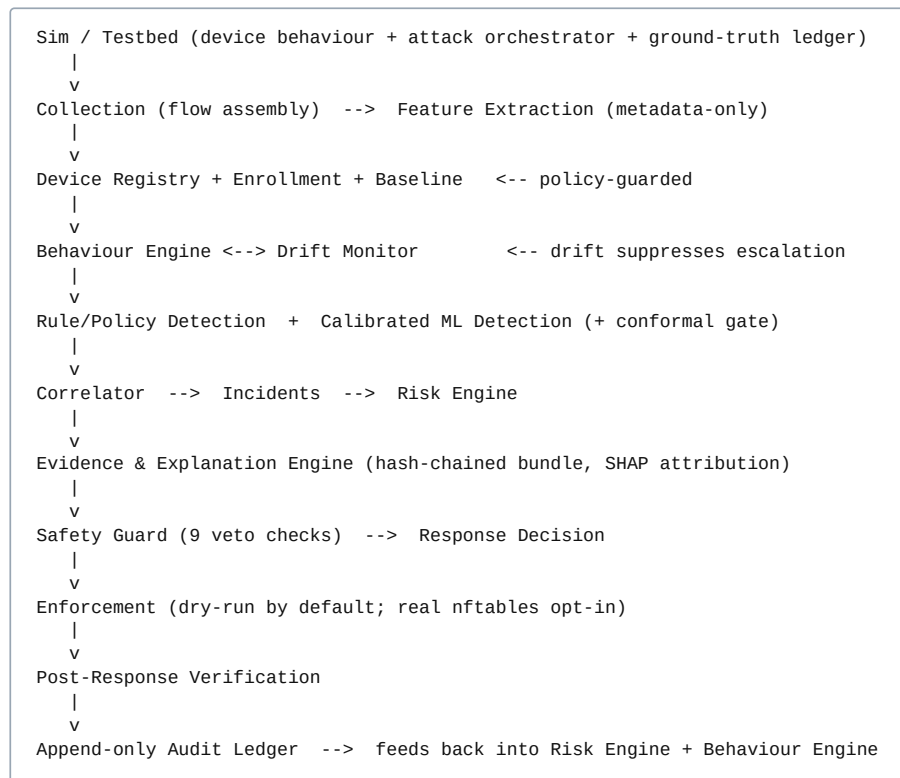


Figure 1. The full ARGUS pipeline. The bottom feedback arrow is what makes this a closed loop rather than a one-shot detect-and-alert pipeline.

Two deployment profiles exist: **dev** (synthetic sim engine + SQLite + FastAPI + console, no special privileges) and an optional real-Linux **live-testbed** profile (network namespaces + veth + real `nftables`, requires `root/CAP_NET_ADMIN`). A third, **production** profile runs on Vercel — see Section 15.

6. End-to-End Workflow

1. **Enrollment.** A device is added to the fleet with a declared type and policy; its baseline is captured.
2. **Traffic arrives** (synthetic generator or real captured packets), assembled into flow records, converted into metadata-only features.
3. **Detection** runs both tracks: rules/policy check the flow against the device's declared policy and known signatures; the calibrated ML detector scores it and produces a conformal prediction set.
4. **Correlation** groups detections within a shared time/entity/chain window into one `Incident`.
5. **Risk scoring** computes a 0–1 score from severity, criticality, confidence, deviation, and blast radius.
6. **Evidence & explanation** builds a hash-chained bundle (feature vector, detections, conformal set, SHAP attribution, risk terms) before any decision is made.
7. **Safety guard** evaluates all nine checks in order; the first failure vetoes the action.
8. **Response decision** picks a tier from the ladder based on risk score, singleton-set status, and drift state; the decision (including which gates passed/failed) is written into the evidence bundle's trace.
9. **Enforcement** applies the action through the configured adapter (dry-run by default) with an expiring TTL.
10. **Post-response verification** checks the outcome at fixed offsets and writes a `Verification` row.
11. **Feedback.** The verification outcome adjusts future risk scoring and behavioural baselining for that device class, closing the loop.
12. **Analyst review (optional, human-in-the-loop).** The console surfaces the incident, its full evidence bundle, and lets an authenticated analyst replay the decision to independently check it was reproducible.

7. Technology Stack

Technology	Purpose	Where Used	Why Used
Python 3.11+	Backend language	<code>argus/</code> , <code>api/</code>	Modern type-hint syntax; mature ML/security ecosystem
FastAPI	ASGI web framework	<code>argus/api/main.py</code> , <code>api/index.py</code>	Async-native, typed request/response models, automatic docs
Uvicorn	ASGI server	local dev server	Standard FastAPI companion server
SQLAlchemy	ORM	<code>argus/db/models.py</code>	Database-agnostic — SQLite by default, Postgres via one env var
SQLite	Default database	local dev	Zero-setup persistence, no separate DB service required
scikit-learn	Isolation Forest, isotonic calibration	<code>argus/detect/ml.py</code>	Established, well-understood anomaly detection on tabular data
SHAP	TreeSHAP attribution	<code>argus/detect/ml.py</code>	Theoretically grounded, per-feature explanation values
river	Online ADWIN drift detector	<code>argus/behavior/</code>	Streaming-appropriate concept-drift detection with formal bounds
pyroute2 / scapy / dpkt	Network namespaces, packet capture/parsing	<code>argus/testbed/</code>	Real Linux networking without needing Docker (which was blocked in the build environment)
nftables	Real enforcement (opt-in)	<code>argus/respond/adapters/</code>	Modern, scriptable Linux firewalling; bridge-family filtering for L2-bridged testbed traffic
React 19 + TypeScript	Frontend framework	<code>console/src/</code>	Type-safe, component-based UI; industry-standard
Vite	Frontend build tool	<code>console/</code>	Fast dev server, production bundling, native TS support
Tailwind CSS v4	Styling	<code>console/src/</code>	Utility-first styling, small production CSS footprint
TanStack Query	Data fetching/caching	<code>console/src/pages/</code>	Polling, caching, and mutation state without hand-rolled fetch logic
React Router	Client-side routing	<code>console/src/App.tsx</code>	Standard SPA routing between Fleet/Incidents/Control
pytest	Backend testing	<code>tests/</code>	45 tests covering every major component, including real network/nftables tests
Vercel	Production hosting	<code>vercel.json</code> , <code>api/</code>	Zero-ops static + serverless hosting for the demo deployment

8. Frontend Deep Dive

The console (`console/`) is a Vite-built React 19 + TypeScript single-page app with three routes.

Fleet (`pages/Fleet.tsx`)

Polls `GET /devices` every 8 seconds via TanStack Query and renders a card grid — device ID, type, state badge, criticality, and a drift indicator with a suppressed-escalation notice when applicable.

Incidents (`pages/Incidents.tsx`)

Polls `GET /incidents` and renders a sortable table; clicking a row opens a full-height drawer (a real ARIA dialog, keyboard-operable, with focus trapped on open and returned on Escape) showing the risk breakdown, the decision trace, the SHAP attribution chart, the evidence bundle's hash fields, and a "Replay decision" button that calls `POST /evidence/{id}/replay` with the saved admin token and displays whether the decision reproduced identically.

Control (`pages/Control.tsx`)

Shows live enforcement mode and kill-switch state (polled every 4 seconds), lets an authenticated user engage/disengage the kill switch and trigger `POST /control/seed-demo` to run the pipeline, and lists active/recent actions with their TTLs.

State & data layer

`lib/api.ts` centralises every backend call behind a typed `api` object with a single relative base path (`/api`), so the same frontend code works unmodified against the local dev proxy and the production same-origin Vercel deployment. `lib/token.ts` stores the admin bearer token in `localStorage` only — a documented demo convenience, not a production auth pattern (Section 12).

9. Backend Deep Dive

The local/dev backend (`argus/api/main.py`) is a single FastAPI app over a SQLAlchemy session dependency. Read endpoints are open; every state-changing endpoint requires a bearer token equal to the `ARGUS_ADMIN_TOKEN` environment variable, checked by a `require_auth` dependency. There is deliberately no default token — the app raises `RuntimeError` at import time if the variable is unset.

Method	Endpoint	Purpose	Input	Output	Auth
GET	<code>/health</code>	Liveness + current enforce/kill-switch state	—	<code>{status, enforce, kill_switch}</code>	None
GET	<code>/control/status</code>	Enforcement mode + kill-switch state	—	<code>{enforce, kill_switch_engaged}</code>	None
POST	<code>/control/kill-switch</code>	Engage/disengage the kill switch	query <code>engage: bool</code>	<code>{kill_switch_engaged}</code>	Bearer token
POST	<code>/control/seed-demo</code>	Runs the full demo pipeline and persists results	—	<code>{seeded, incidents, bundles, actions}</code>	Bearer token
GET	<code>/devices</code>	List all enrolled devices	—	array of device objects	None
GET	<code>/incidents</code>	List incidents with joined risk/evidence/action/verification summary	—	array of incident objects	None
GET	<code>/evidence/{bundle_id}</code>	Full evidence bundle	path <code>bundle_id</code>	evidence bundle object	None
POST	<code>/evidence/{bundle_id}/replay</code>	Re-runs the decision path and compares to the stored outcome	path <code>bundle_id</code>	<code>{reproduced, original_action, replayed_action, ...}</code>	Bearer token
GET	<code>/actions</code>	List all enforcement actions taken	—	array of action objects	None

CORS is enabled with a wildcard origin for local development convenience; the production API (Section 15) is same-origin with the static console, so this is not exposed there in practice.

10. Database

SQLite by default (`ARGUS_DATABASE_URL` can point at Postgres instead — the ORM layer doesn't change). Seven tables, managed entirely through SQLAlchemy's declarative models (`argus/db/models.py`); no raw SQL anywhere in the codebase.

Table	Key columns	Relationship
<code>devices</code>	<code>device_id</code> (PK), <code>device_type</code> , <code>criticality</code> , <code>state</code> , <code>is_drifting</code> , <code>baseline_version</code>	Referenced by <code>device_id</code> from <code>incidents/actions</code>
<code>incidents</code>	<code>incident_id</code> (PK), <code>device_id</code> , <code>chain_position</code> , <code>agreement_score</code> , <code>scenario</code> , <code>status</code>	Parent of <code>risk_assessments</code> , <code>evidence_bundles</code>
<code>risk_assessments</code>	<code>id</code> (PK), <code>incident_id</code> (FK), <code>score</code> , <code>terms</code> (JSON)	Child of <code>incidents</code>
<code>evidence_bundles</code>	<code>bundle_id</code> (PK), <code>incident_id</code> (FK), <code>device/feature_vector/detection/baseline/risk/decision</code> (JSON), <code>trace</code> (JSON), <code>prev_bundle_hash</code> , <code>merkle_root</code>	Child of <code>incidents</code> ; parent of <code>actions</code>
<code>actions</code>	<code>action_id</code> (PK), <code>bundle_id</code> (FK), <code>device_id</code> , <code>tier</code> , <code>action</code> , <code>dry_run</code> , <code>applied_at</code> , <code>reverted_at</code> , <code>ttr_seconds</code>	Child of <code>evidence_bundles</code> ; parent of <code>verifications</code>
<code>verifications</code>	<code>id</code> (PK), <code>action_id</code> (FK), <code>outcome</code> , <code>checked_at</code> , <code>offset_seconds</code>	Child of <code>actions</code>
<code>audit_log</code>	<code>seq</code> (PK), <code>ts</code> , <code>actor</code> , <code>event_type</code> , <code>payload</code> (JSON)	Independent append-only ledger

```
devices —1:N— incidents —1:N— risk_assessments
                        |
                        —1:N— evidence_bundles —1:N— actions —1:N— verifications

audit_log (independent, append-only)
evidence_bundles.prev_bundle_hash chains each bundle to the one before it
```

Figure 2. Entity relationships. All writes go through the pipeline (`argus/pipeline.py`) or the FastAPI endpoints — there is no direct external write path into the database.

CRUD is almost entirely create-and-read: `devices/incidents/bundles/actions/verifications` are created once by the pipeline and read by the API; the only update-like operations are the kill switch's in-memory state and a device's `is_drifting` / `state` flags, both written by their owning component, never by an external caller directly.

11. AI / ML Component

11.1 Why ML is used at all

Rules only catch attack shapes someone thought to author a rule for. The ML track exists specifically to give coverage against attacks that don't match a known signature — the calibrated Isolation Forest's job is exclusively that gap.

11.2 Features

The seven metadata-only feature groups from Section 4.3 (volume/shape, timing, periodicity, destination behaviour, DNS, TLS/JA4, policy) — the same features both detection tracks consume.

11.3 Preprocessing

Feature extraction is per-device-type; the Isolation Forest is trained per device type rather than globally, since normal traffic shape differs enormously between, say, a smart plug and an IP camera. No feature scaling beyond what Isolation Forest's tree-splitting already tolerates is required.

11.4 Algorithm

Isolation Forest (Liu, Ting & Zhou, ICDM 2008) — an ensemble of random partitioning trees; anomalies are isolated in fewer splits than normal points, giving an anomaly score without needing labelled attack data. Raw scores are **isotonic-calibrated** on a held-out split so they behave as genuine probabilities, then wrapped in an **inductive conformal prediction** gate (Papadopoulos, 2008) that outputs a prediction set with a distribution-free coverage guarantee at $\alpha=0.05$, rather than thresholding the raw score.

11.5 Training and calibration

Training and calibration currently use the labelled data available from the calibration split the pipeline itself produces (synthetic/live-testbed scale — dozens of rows per device type), not a large public dataset; the public-dataset track (Section 20) was verified infeasible to reach from the build environment.

11.6 Inference

At detection time: extract features → score with the calibrated Isolation Forest → compute the conformal prediction set at $\alpha=0.05$ → compute TreeSHAP attribution from a companion `RandomForestClassifier` trained on the same calibration split → all three outputs (calibrated score, conformal set, attribution) are written into the evidence bundle before any response decision is made.

11.7 Evaluation (measured, not projected)

Evaluated via a 9-configuration ablation (A0–A8) at 5 seeds against a genuine benign holdout, using Mann-Whitney U, Holm-Bonferroni correction, and Cliff's delta. The headline measured result: **A6** (identical detection stack to the full system, by construction, with only the response ladder disabled) shows **0% containment** against the full system's **100%** (5/5 seeds) — despite having *identical* detection metrics. This is direct, measured evidence that classification accuracy is a weak predictor of whether a system actually contains an attack — see the accompanying IEEE paper for the full statistical treatment.

No fabricated numbers: every figure in this section traces to a named script or test (`eval/harness.py` , `eval/ablation.py` , `tests/test_ablation.py`) and the commit referenced in `decisions.md` . Where a number doesn't exist (e.g. cross-dataset generalisation, a ≥ 100 -bundle replay sample), Section 20 says so explicitly rather than estimating one.

11.8 Known limitations of the ML component

- Trained/calibrated on a small, synthetic/live-testbed-derived split, not a large public dataset.
- No cross-dataset generalisation test has been run (Section 20).
- Risk-engine weights that combine with the ML score are authored, not learned.

12. Security

12.1 Implemented

IMPLEMENTED	Bearer-token authentication on every state-changing endpoint, with no default token (fails loudly at startup if unset).
IMPLEMENTED	Dry-run enforcement by default everywhere (<code>ARGUS_ENFORCE=false</code>); live enforcement is opt-in.
IMPLEMENTED	Kill switch checkable through three independent mechanisms (file, env var, API).
IMPLEMENTED	Protected-device allowlist that a risk score can never override.
IMPLEMENTED	Rate limiting on ARGUS's own actions, per device and network-wide.
IMPLEMENTED	Auto-expiring rollback timer on every applied action.
IMPLEMENTED	Hash-chained, tamper-evident evidence bundles (SHA-256, chained via <code>prev_bundle_hash</code>).
IMPLEMENTED	No payload inspection anywhere, as a permanent architectural commitment (metadata-only feature extraction).
IMPLEMENTED	Secrets excluded from the repository (<code>.env</code> gitignored, <code>.env.example</code> placeholder only).
IMPLEMENTED	CI-independent linting (<code>ruff</code>) kept clean.

12.2 Recommended Future Security Improvements

FUTURE	Replace the console's <code>localStorage</code> admin-token pattern with a real session/OAuth flow — explicitly documented in the UI itself as "a demo convenience, not a production auth flow."
FUTURE	Per-user roles/RBAC instead of a single shared admin token.
FUTURE	Restrict CORS from the current wildcard origin to a specific allowlist for any non-demo deployment.
FUTURE	TLS termination and secret management appropriate to a real production deployment (the current Vercel deployment relies on the platform's own TLS and a single project environment variable).
FUTURE	Real Suricata/Zeek signature integration in place of the documented threshold stand-in in <code>argus/detect/rules.py</code> .
FUTURE	Audit-log export/retention policy for long-running deployments (currently append-only in the same database, no rotation).

13. Important Code Concepts

Data-flow contracts between stages

Every pipeline stage consumes only the typed schema the previous stage produced — never a neighbour's internal state. This is what makes the ablation harness possible: a stage becomes a pass-through by toggling a parameter, without touching any other stage's code, and it's also what lets the live-packet testbed and the synthetic engine feed the identical downstream pipeline with zero code changes.

Fail-loud configuration

`ARGUS_ADMIN_TOKEN` has no default anywhere in the codebase; both the local and production API endpoints raise `RuntimeError` at import time if it's unset, rather than silently running with an open admin surface.

Dependency injection for the database session

FastAPI's `Depends(get_db)` pattern yields a session per request and guarantees it's closed afterward, even on an exception — standard FastAPI/SQLAlchemy practice used throughout `argus/api/main.py`.

Simulated-clock threading (the bug the replay mechanism found)

The action rate limiter originally gated on wall-clock time. A full simulated pipeline run spans several simulated *hours* of device behaviour while executing in well under a real second, so every action in a single run looked, to a wall-clock-gated limiter, like a burst within the same real-world hour — spuriously tripping the rate limit. The fix threads a `now: float | None` parameter through the guard and ladder functions, using each incident's own simulated timestamp instead of defaulting to `time.time()`. This is a genuine example of a subtle correctness bug that only the replay mechanism's own design (compare a fresh re-run against a stored decision) was positioned to catch.

Adapter interfaces for swappable behaviour

Enforcement adapters share one interface (`apply`, `revert`, `list_active`), so `DryRunAdapter`, `NoOpAdapter` (the ablation's detection-only baseline), and the real `nftables` adapter are interchangeable without any change to `argus/respond/ladder.py`'s calling code.

14. API / Data Flow

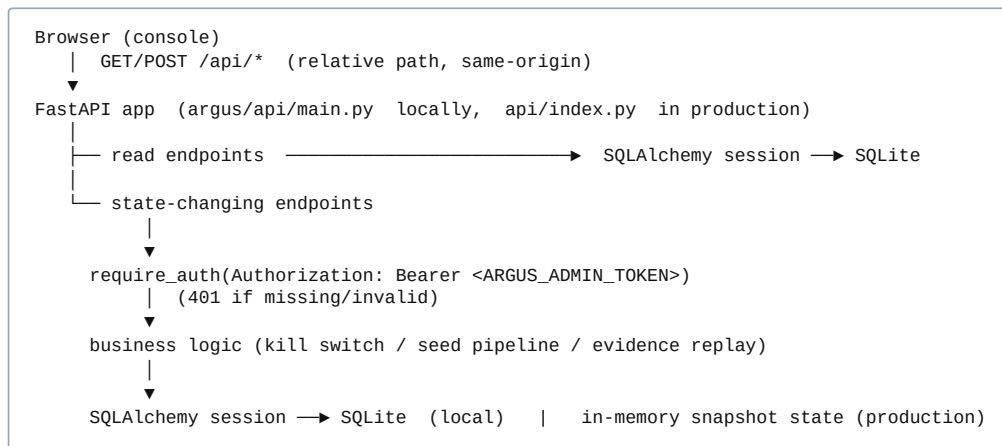


Figure 3. Request path for both local and production deployments. The console's `lib/api.ts` always calls a relative `/api/*` path, so the same frontend bundle works against either backend unmodified.

15. Deployment Architecture

Two real, running deployment shapes exist for this project:

Local

Uvicorn serving `argus/api/main.py` on port 8000, backed by a SQLite file on disk; Vite's dev server on port 5173 proxying `/api/*` to the backend (stripping the prefix). Full ML pipeline, live-testbed extra, and real `nftables` enforcement are all available here.

Production (Vercel)

The console is built to static files (`console/dist`) and served directly by Vercel. A separate, deliberately lighter FastAPI endpoint (`api/index.py`) runs as a Python serverless function, importing only the pure-stdlib pieces of the real codebase (`argus.evidence.bundle`, `argus.evidence.replay`, `argus.respond.guard`) — no `numpy/scikit-learn/shap` in the production dependency set. It serves a real, pre-computed snapshot of one actual pipeline run (`api/seed_snapshot.json`, produced by `scripts/export_seed_snapshot.py`) rather than fabricated demo data, while running evidence replay and the kill switch live and unmodified on every request. `vercel.json`'s rewrite rule routes every `/api/*` request to this one function. See Section 17 for exactly why the full system can't all run serverless, and `docs/06-vercel-deployment.md` for the complete reasoning.

16. Local Setup Guide

Exact commands (see `README.md` for the full version with a troubleshooting table):

```
git clone <repo-url> argus && cd argus
make install                # venv + pip install -e ".[dev]" + npm install
cp .env.example .env        # then edit: set ARGUS_ADMIN_TOKEN
make test                   # 45 tests, ~65s

# terminal 1
export $(cat .env | xargs) && make api      # http://localhost:8000
# terminal 2
make console                          # http://localhost:5173
```

In the console's Control tab: paste the token from `.env`, save it, then click "Run demo pipeline" — this genuinely executes the full loop and persists the result to SQLite; nothing is canned.

17. Production Deployment Guide

Vercel project settings actually used: build command `cd console && npm install && npm run build`, output directory `console/dist`, one required project environment variable, `ARGUS_ADMIN_TOKEN` (the production function fails to start without it, identical fail-loud behaviour to local). `vercel.json` defines the build command, output directory, and the rewrite that routes all `/api/*` traffic to the single `api/index.py` function.

Three real platform constraints keep production narrower than local, stated explicitly rather than hidden: (1) no `root/CAP_NET_ADMIN` in a serverless sandbox, so the live network-namespace testbed cannot run there at all; (2) no persistent filesystem across separate function invocations, so a live SQLite "seed then browse" flow doesn't hold up — production instead serves a real, pre-computed snapshot; (3) function size/time budget — `numpy/scikit-learn/shap` are not part of the production dependency set, so on-demand ML fitting doesn't happen there either. What stays genuinely live in production regardless: evidence replay and the kill switch, both pure-stdlib code with no ML dependency.

To refresh the production demo data with a new real pipeline run: `python scripts/export_seed_snapshot.py`, then commit the regenerated `api/seed_snapshot.json` and redeploy.

18. Testing

45 pytest tests, actually run, not estimated (`pytest tests/ -v` , ~65 seconds). Coverage includes: feature-extraction fixtures with hand-verified expected values; enrollment and policy-based refusal; the full detection → correlation → risk integration path; each of the five safety-guard mechanisms exercised directly and independently; evidence hash-chain tamper detection; decision replay; six tests against the real network-namespace testbed (fabric setup/teardown leaving no residual state, real UDP delivery between namespaces, captured real attack flows checked against wall-clock ground truth, the containment check); four tests against the real `nftables` adapter (isolate blocks and revert restores an actual socket connection; `block_destination` verified narrower than `isolate` against two real targets; a kill-switch-style flush restores connectivity); SHAP attribution tests; ablation-mechanism tests (asserting A6's detection metrics equal A0's by construction); evaluation-statistics unit tests; and dataset-subsampling tests against a synthetic fixture.

Frontend: `tsc -b` (full TypeScript typecheck) runs as part of every production build; the console was also verified live against the real backend with browser automation, including clicking the actual "Replay decision" button and confirming a real accessibility pass.

19. Troubleshooting

Symptom	Cause	Fix
<code>RuntimeError: ARGUS_ADMIN_TOKEN is not set</code>	<code>.env</code> not created/loaded	<code>cp .env.example .env</code> , edit, <code>export \$(cat .env xargs)</code>
Console: "Could not reach the API"	Backend not running	Start <code>make api</code> in a separate terminal
<code>npm install</code> fails (native binding / EBADENGINE)	Node < 20.19	Upgrade Node (<code>nvm install 22</code>)
401 on Replay/Kill Switch/Seed Demo	Token missing or mismatched in the console	Control tab → paste exact <code>.env</code> token → Save
<code>pytest</code> failures mentioning <code>pyroute2/scapy</code>	Optional live-testbed extras not installed	<code>pip install -e ".[live-testbed]"</code> , or ignore — the core suite doesn't need them
<code>ModuleNotFoundError: No module named 'argus'</code>	Running Python outside the project venv	Use <code>.venv/bin/python</code> , or re-run <code>make install</code>

20. Limitations

- **Scenario coverage:** only 2 of the 7 implemented attack scenarios (`mirai`, `low_and_slow`) run on the live-packet testbed and in the ablation harness; the other five run correctly through the synthetic pipeline only.
- **Dataset track:** no public-dataset numbers (CICIoT2023, IoT-23) appear anywhere in this project. The outbound path to fetch them returned HTTP 403 in the build environment — verified directly, not assumed — the same failure mode independently confirmed for Docker Hub. The subsampling/parity logic is implemented and unit-tested against a synthetic fixture, ready the moment a reachable environment is available.
- **Evidence-replay sample size:** 100% reproducibility is measured on 19 bundles from one seeded run; a substantially larger multi-run sample (target: ≥ 100 bundles) is needed before this is a stable claim rather than an early, favourable measurement.
- **SHAP training data:** trained on the same small calibration split the conformal detector uses, not a large public dataset, for the same reachability reason as above.
- **No human evaluation of explanations:** the original protocol (participants shown evidence bundles, scored against ground truth) needs real human subjects an autonomous/solo build effort cannot supply; preserved as explicit future work.
- **Live-testbed traffic realism:** real packets/sockets/namespaces, but not full application-layer protocol implementations (no real MQTT broker, no real TLS handshake) — metadata-realistic, not protocol-complete; costs nothing feature-wise since ARGUS is metadata-only by design.
- **Risk weights are authored, not learned,** defended by a stated sensitivity analysis rather than beyond scrutiny.
- **Production deployment scope** is narrower than local by platform necessity (Section 17) — this is disclosed in the deployed app's own `/api/health` response (`mode: "production-snapshot"`), not hidden.

21. Future Enhancements

PROPOSED	Port the five synthetic-only attack scenarios into the live-packet testbed and the ablation's scenario loop.
PROPOSED	Run the evidence-replay mechanism against ≥ 100 bundles across multiple seeded runs.
PROPOSED	Run the public-dataset track (CICIoT2023 training, IoT-23 cross-dataset generalisation) once a network-unrestricted environment is available.
PROPOSED	Conduct the human evaluation of explanation quality, if the project continues with human collaborators.
PROPOSED	Real Suricata/Zeek signature integration in place of the current threshold stand-in.
PROPOSED	Learned (rather than authored) risk-engine weights, validated against the existing sensitivity analysis as a baseline.
PROPOSED	Per-user roles/RBAC and a real session-based auth flow for the console, replacing the current single shared admin token.

22. Resume Explanation

30-second version

"I built ARGUS, a closed-loop IoT intrusion detection system that doesn't just alert on an attack — it decides whether to act, actually blocks the device on a real network, proves afterward why it acted with a tamper-evident evidence trail, and measures whether the response actually worked. The headline result I measured: a version of the system with identical detection accuracy to the full system, but with response disabled, had zero successful containments versus the full system's 100% — direct proof that detection accuracy alone doesn't tell you if a system actually works."

1-minute version

"Most published IoT intrusion detection research stops at classification and is evaluated offline — a recent systematic review found 29 of 32 studies never tested mitigation at all. I built ARGUS to close that gap: it runs a full detect-correlate-risk-score-explain-decide-enforce-verify loop, gates any autonomous action behind an inductive conformal prediction check I implemented from scratch, and binds every decision to a hash-chained evidence bundle that can be independently replayed to check it's reproducible — which it is, on every bundle I've tested. When my planned Docker-based testbed turned out to be blocked in my build environment, I built a real alternative from first principles — actual Linux network namespaces carrying real packets — rather than falling back to pure simulation, and along the way found and fixed two genuine kernel-networking bugs that only showed up once I exercised the real path. The core measured finding, from a 9-configuration ablation study with proper statistics, is that a system with detection metrics identical to the full one by construction showed zero containment versus the full system's 100% — evidence that the field's standard metric doesn't measure what actually matters."

2-minute version

Combine the 1-minute version with: the safety-guard design (nine independent veto checks including a kill switch checkable three separate ways, dry-run-by-default, and an auto-expiring rollback timer on every action — because an autonomous system that can act on a network needs to be trivially, redundantly stoppable); the real `nftables` enforcement work and the specific bridge-netfilter/Docker-FORWARD-policy bug found while building it; the full-stack delivery (FastAPI + SQLAlchemy backend, React/TypeScript console with a real accessibility pass verified by browser automation); and close with the honest limitations list (Section 20) as evidence of engineering judgment, not a weakness to hide — "I know exactly what isn't done yet and why, which is why I can tell you precisely what I'd build next."

23. Viva / Interview Preparation

Q: Why didn't you just build a better classifier?

Because the field's actual gap, per the gap-analysis I cite, isn't classifier quality — it's that most systems never act on what they detect. My own ablation proves the point directly: identical detection, completely different outcome, once you measure containment instead of just F1.

Q: What exactly does "evidence-bound" mean, concretely?

Every autonomous decision is written into a bundle containing the feature vector, which detectors fired, the conformal prediction set, SHAP attribution values, the risk score's component terms, and the safety guard's full pass/fail trace, before the action is taken — and it's hash-chained to the previous bundle so tampering with history is detectable. Then there's a replay mechanism that independently re-derives the decision from those stored values and checks it matches. That's the difference between "evidence" and "a log message."

Q: What is conformal prediction and why use it here instead of a confidence threshold?

A raw softmax/probability output from a classifier isn't a formal guarantee about any individual prediction — it's just a number that happens to often behave reasonably. Inductive conformal prediction instead produces a set of plausible labels with a distribution-free, finite-sample guarantee that the true label is excluded with probability at most α . I gate autonomous escalation on that set being a singleton — i.e., the model is confident in the sense the guarantee actually backs, not just confident-looking.

Q: What happens if the model is wrong and the system blocks a legitimate device?

Every action has an auto-expiring rollback timer, so the default state is always restored unless the action is actively renewed. Post-response verification checks specifically for collateral damage (something legitimate breaking) and, if found, raises the action threshold for that device class going forward. And the kill switch can stop everything through three independent paths if a human needs to intervene immediately.

Q: Your Docker testbed plan didn't work — walk me through what you actually did.

Image pulls from Docker Hub returned a 403 in my build environment — I verified that directly rather than assuming it. Rather than fall back to synthetic-only, I built a real alternative: Linux network namespaces and veth pairs on a real bridge, with a custom multi-interface scapy sniffer capturing genuine packets into a pcap, parsed into the exact same flow schema my synthetic engine produces — so the rest of the pipeline needed zero changes. Building it surfaced two real kernel-networking bugs: Docker's own iptables ruleset was silently absorbing my bridge's traffic via a sysctl that affects any L2-bridged traffic system-wide, and once I fixed that, my nftables rules needed to move to the bridge table family specifically to see bridged traffic at all.

Q: How did you validate your enforcement actually works, not just that the command didn't error?

I never trusted `nft`'s exit code alone. I tested against real socket connections: a connection that provably fails while isolation is active, and provably succeeds again after either a targeted revert or a kill-switch-style full flush — and I specifically verified `block_destination` is narrower in effect than full `isolate` against two distinct real targets, matching what the design intends, not just what the code claims.

Q: What's the biggest limitation you'd tell a reviewer about unprompted?

The evidence-replay reproducibility number — 100% — is measured on only 19 bundles from one run. The mechanism is proven, but the sample is small; I'd want at least 100 bundles across multiple seeded runs before I'd call that a stable property rather than an early, favourable measurement. I say that upfront rather than waiting to be asked.

Q: Why SQLite instead of Postgres?

Zero-setup local development — `make seed` shouldn't require a running database container. SQLAlchemy's ORM layer is the actual portability boundary, not the specific engine; pointing `ARGUS_DATABASE_URL` at Postgres is a one-line config change with no code change.

Q: How is the production (Vercel) deployment different from local, and why?

Three real platform constraints: no `root/CAP_NET_ADMIN` in a serverless sandbox, so the live network-namespace testbed can't run there at all; no persistent filesystem across separate function invocations, so a live SQLite flow doesn't hold up; and a function size/time budget that excludes numpy/scikit-learn/shap from the production dependency set. So production serves a real, pre-computed snapshot of one actual pipeline run instead of re-simulating live, while evidence replay and the kill switch — both pure-stdlib, no ML dependency — run genuinely live on every request. That split, and why, is documented in the code itself, not hidden.

Q: What would you build next if you had another month?

In priority order, matching my own roadmap: port the remaining five attack scenarios into the live-packet testbed and the ablation; run evidence replay against a much larger bundle sample; run the public-dataset track the moment I have unrestricted network access, since that's the piece needed for a defensible cross-dataset generalisation claim; and, if I had access to willing participants, run the human evaluation of explanation quality my evaluation protocol already specifies but couldn't execute alone.